KATALEYA

Founder's Technical Reference

v2.3 — March 2026 | Confidential | github.com/kwasikontor45/kataleya

What this document is

A plain-English guide to every architectural decision in the codebase — written so you can explain, defend, and understand your own app in any room. Investor due diligence. CTO conversations. App Store review. Users asking about privacy. This document prepares you for all of it.

Seven Stories

- Story 1: The Vault — how data is stored and why three systems
- Story 2: A Mood Log — what happens from tap to database
- Story 3: The Burn Ritual — what actually gets erased
- Story 4: The Encrypted Backup — what the cryptography does
- Story 5: The Sponsor Channel — what the server sees and never sees
- Story 6: The Insights Engine — patterns without surveillance
- Story 7: End-to-End Encrypted Messages — how the crypto works

— CURRENT BUILD STATE — V2.3 —

What Is Actually Shipped

This is the honest, complete state of the codebase as of March 2026. Every row has been verified against the live source files on GitHub (github.com/kwasikontor45/kataleya).

Feature	Status
Three-vault storage (Surface / Sanctuary / Fortress)	✅ **Complete**
SQLite schema + migration guard	✅ **Complete**
Circadian color engine (4 phases, adaptive polling)	✅ **Complete**
Orchid SVG with accelerometer sway + RMS restlessness	✅ **Complete**
Sobriety timer (second-precise) + 10 milestones	✅ **Complete**
Haptic bloom profiles (Budding → Parting → Labellum → Full Bloom)	✅ **Complete**
Mood logging (5 states, circadian phase, restlessness wired)	✅ **Complete**
Private journal (4,000 chars, delete-on-demand)	✅ **Complete**
Growth screen (milestones + local ML insights)	✅ **Complete**
Insights engine (3 patterns, confidence-gated ≥ 72%)	✅ **Complete**
Insights UI (data age + sufficiency disclosure)	✅ **Complete**
Vault screen + encrypted backup UI	✅ **Complete**
AES-256 backup (native OpenSSL, timingSafeEqual, zero-knowledge)	✅ **Complete — needs EAS build**
Burn Ritual (confirmation gate + tombstone crash recovery)	✅ **Complete**
Bridge screen + tombstone recovery at launch	✅ **Complete**
Onboarding (7 steps: welcome → name → substance → date → privacy → notifs → enter)	✅ **Complete**
Push notifications (daily reminder + milestones, cancel-before-schedule)	✅ **iOS complete / Android needs EAS**
Sponsor channel (invite / accept / poll / presence / check-in)	✅ **Complete**
Sponsor haptics (Water ripple, Light 72-BPM heartbeat)	✅ **Complete**
Offline check-in queue + 24h TTL replay	✅ **Complete**
Presence signal log (per-type, clearable)	✅ **Complete**
E2E encrypted sponsor messages (X25519 + XSalsa20-Poly1305)	✅ **Complete**
API server rate limiting (per-token, 4 endpoint maps, GC)	✅ **Complete**

Cover / panic screen	✅ **Complete**
MorningBloom contrast fix (dawn phase readability)	✅ **Complete**
Subscription / payment layer	➡️ **v3.0 — deferred**

— ARCHITECTURE OVERVIEW —

How Kataleya Is Built

Kataleya is a React Native mobile app built on Expo SDK 52. The monorepo has two deployable pieces: the mobile client and a minimal relay server.

```
kataleya/                          ← github.com/kwasikontor45/kataleya
  artifacts/
    kataleya-app/                  ← the mobile app
      app/                         ← screens (Expo Router)
        (tabs)/                    ← five main tabs
          index.tsx                ← sanctuary (home)
          journal.tsx              ← mood logging + journal
          growth.tsx               ← milestones + insights
          vault.tsx                ← backup + burn ritual
          sponsor.tsx              ← sponsor channel + messages
        bridge.tsx                 ← opening ritual + tombstone recovery
        onboarding.tsx             ← 7-step onboarding
        burn.tsx / cover.tsx       ← burn route + panic screen
      hooks/                       ← business logic
        useCircadian.ts            ← phase + theme engine
        useInsights.ts             ← local pattern detection
        useNotifications.ts        ← push notification orchestration
        useOrchidSway.ts           ← accelerometer + restlessness RMS
        useSobriety.ts             ← timer + milestone computation
        useSponsorChannel.ts       ← channel + messages + offline queue
        useResponsiveHeart.ts      ← BPM 45-78 engine, system state word
      utils/                       ← data layer
        db.ts                      ← SQLite init + migration
        storage.ts                 ← three-vault API (491 lines)
        backup.ts                  ← encrypted backup (native crypto)
        crypto.ts                  ← E2E crypto (tweetnacl, 55 lines)
        hapticBloom.ts             ← haptic profiles
        sanitize.ts                ← input cleaning
    api-server/                    ← relay server (sponsor channel only)
      src/routes/sponsor.ts        ← all channel + message endpoints
```

The golden rule

No file in the codebase calls AsyncStorage, SQLite, or SecureStore directly. Every data operation goes through Surface, Sanctuary, or Fortress — the three exports from utils/storage.ts. This is how the privacy guarantee is enforced in code, not just in policy.

— DATA ARCHITECTURE —

The Three Vaults

Vault	Engine	What lives here	Why this engine
Surface	AsyncStorage	Name, sobriety date, onboarding, notif prefs, burn tombstone, pending check-in, presence log, dark mode override	Fast key-value. OS file encryption. No sensitive health data.
Sanctuary	SQLite	Mood logs, journal entries, circadian event log	Real database. Lives entirely on-device. No network port. No user_id column.
Fortress	SecureStore / Keychain	Sponsor channel creds + E2E crypto keys (public, secret, peer public) — 8 keys total	Hardware-backed: iOS Secure Enclave, Android Keystore. Private keys never leave this vault.

Fortress now holds 8 keys — the original 5 channel credential keys plus 3 new E2E crypto keys added in v2.3: kataleya.fortress.crypto_pk (own public key), kataleya.fortress.crypto_sk (own private key), kataleya.fortress.peer_pk (partner's public key). All 8 keys are deleted atomically by Fortress.clear() on disconnect, using Object.values(FK) — no manual list to maintain.

— STORY 1 —

The Vault

How to say it

"We use three separate storage systems — a notepad, a local database, and a hardware keychain — because different data has different risk profiles, and separating them means a mistake in one place can't compromise the others."

The one-sentence version of each vault

Surface — the notepad

AsyncStorage. Holds preferences and state: your display name, sobriety start date, onboarding completion, notification settings, burn tombstone (crash recovery), pending check-in (offline resilience), presence signal log. Nothing health-sensitive. OS-encrypted at file level.

Sanctuary — the filing cabinet with no network port

SQLite. Holds every mood log, journal entry, and circadian event. SQLite is a database engine that runs entirely on the device — it physically cannot send data anywhere. The schema has no user_id column. CHECK constraints enforce data integrity at the database level, not the app level.

Fortress — the safe deposit box

SecureStore. On modern iPhones this routes through the Secure Enclave — a physically separate chip. Holds 8 keys: 5 sponsor channel credentials and 3 E2E crypto keys. The private key for message encryption lives exclusively here. Fortress.clear() wipes all 8 keys atomically on disconnect.

Q: "How do you know health data never leaves the device?"

A: "Because the only code that can write health data is Sanctuary, and Sanctuary has no fetch() calls. Search utils/storage.ts for fetch — zero results. The vault separation makes it technically impossible to accidentally route health data to a network call without it being immediately visible in code review."

— STORY 2 —

A Mood Log

Five systems touch the moment a user taps a mood state. Here is exactly what happens.

Step 1: The circadian phase is already known

useCircadian.ts runs continuously. It reads the device clock and maps it to one of four phases: dawn (5am–8am), day (8am–5pm), golden hour (5pm–8pm), night (8pm–5am). During transitions it polls every 10 seconds. During stable phases it waits until the next minute boundary to save battery. No network call. No server.

Step 2: The accelerometer has been sampling for 5 seconds

useOrchidSway.ts samples at 50ms intervals (20 samples/second). Each sample computes the delta magnitude — how much the acceleration changed since the last reading. These deltas fill a rolling buffer of 100 samples (5 seconds). Every second, the hook computes the Root Mean Square of the buffer and normalizes it to 0.0–1.0, where 2.0 m/s² maps to 1.0. This is the restlessness score. A pacing user scores higher than a still user. If no accelerometer, the score is 0.

Step 3: The user selects a mood and taps save

```
// journal.tsx (simplified)
const { restlessnessScore } = useOrchidSway();
const { phase } = useCircadian();

await Sanctuary.saveMoodLog({
  ts: Date.now(),
  moodScore: selectedScore,      // 1-10
  circadianPhase: phase,          // 'dawn' | 'day' | 'goldenHour' | 'night'
  restlessness: restlessnessScore, // 0.0-1.0, live from accelerometer
  context: contextText || undefined,
});
```

Step 4: The database write

Sanctuary.saveMoodLog() generates a unique ID (timestamp base-36 + random segments), runs a parameterized INSERT into SQLite, and logs a circadian event ('mood_log' at this timestamp and phase). WAL mode means reads don't block during the write. The CHECK constraint on mood_score is enforced at the database level.

Step 5: The notification is suppressed

After a successful save, suppressReminderForToday() cancels any pending daily reminder. The user just logged — they don't need a nudge. The reminder reschedules itself on the next app open.

The server sees nothing

Zero network calls happen during a mood log. No analytics event. No telemetry. The data goes directly to SQLite.

— STORY 3 —

The Burn Ritual

A complete, irreversible wipe of all three vaults in four seconds. Here is how it works.

The confirmation gate

The user selects a reason (four options, including 'no reason / private'). They hold the ignite button for 3 seconds. When the hold completes, a confirmation screen appears: 'This will permanently erase everything. There is no undo.' Two buttons: Erase everything and Cancel. The wipe does not start until Erase everything is tapped.

Phase 1: The tombstone

Before any data is touched, Surface.writeBurnTombstone() writes kataleya.surface.burn_log with a timestamp and confirmed:true. This is the crash recovery signal. If the OS kills the process mid-wipe, this key survives and triggers completion on next launch.

Phase 2: Parallel native vault wipes

Sanctuary.wipeSQLiteData() and Fortress.clear() run in parallel (Promise.all). They are independent. Fortress.clear() deletes all 8 keys — 5 channel credentials and 3 crypto keys — via Object.values(FK). Neither touches Surface, so the tombstone survives this phase.

Phase 3: Surface wipe — last

Only after both native vaults confirm empty does Surface.clearAll() run. This erases everything including the tombstone. A clean burn leaves no trace.

```
// BurningRitual.tsx — executeBurn()
await Surface.writeBurnTombstone();          // Phase 1: crash signal

await Promise.all([                          // Phase 2: parallel
  Sanctuary.wipeSQLiteData(),                // clears 3 SQLite tables
  Fortress.clear(),                          // deletes all 8 Keychain keys
]);

await Surface.clearAll();                    // Phase 3: last — erases tombstone
```

Tombstone crash recovery

bridge.tsx checks Surface.getBurnTombstone() before routing anywhere. If found: re-runs Phase 2 (both wipes are idempotent on empty vaults), then Phase 3, then routes to onboarding. The bridge animation plays during this — the recovery is invisible to the user.

— STORY 4 —

The Encrypted Backup

The plain-English version of the cryptography

PBKDF2-SHA256 runs the passphrase through 100,000 SHA-256 computations to produce a 256-bit key. At a billion guesses per second, a 12-character random passphrase would take longer than the age of the universe to crack. AES-256-CBC encrypts the data. HMAC-SHA256 authenticates it. The MAC is checked before decryption is attempted — wrong passphrase is caught immediately via timingSafeEqual(), a constant-time comparison that prevents timing attacks.

The native crypto advantage

react-native-quick-crypto is a JSI native module. JSI means JavaScript calls directly into native code — OpenSSL on Android, CommonCrypto on iOS. Key derivation runs in native memory for 100,000 iterations without expanding through the JavaScript heap. The RNG is /dev/urandom on iOS and SecureRandom on Android. This requires a native EAS build — Expo Go cannot load JSI modules.

The blob format — frozen

```
{ v, alg, iters, salt, iv, mac, data }
// iters read from blob at decrypt time — old backups at 100k still restore
// data = base64 ciphertext. No plaintext fields anywhere.
```

What cloud providers see

The blob is created entirely on-device before the share sheet opens. iCloud and Google Drive receive encrypted bytes only. Without the user's passphrase — which we never see — the content is unrecoverable by anyone.

— STORY 5 —

The Sponsor Channel

What the server holds — complete list

Field	What it is	What it is NOT
checkedInDate	"Mon Mar 28 2026" presence string	Not a mood score
orchidStage	"seedling" — milestone label	Not sobriety data
milestones	An integer count	Not journal content
presenceQueue	type: water or light only	No health context
messageQueue	ciphertext + nonce only	Server cannot decrypt
userPubKey / sponsorPubKey	X25519 public key — 32 bytes	Not the private key

The server has no user database

Channel routing state is persisted to data/channels.json using atomic writes (temp file + rename, debounced 800ms) so server restarts are invisible to users. Message queues and presence signals are volatile — not persisted. No logs, no records of signal content. The message queue holds opaque ciphertext — the server cannot read any message content.

What the server never sees

Search useSponsorChannel.ts for any of these — zero results: moodScore, journal body, sobriety start date, substance, name, restlessness, private key.

The offline queue

Check-ins queue to Surface (kataleya.surface.pending_checkin) on network failure. Every foreground event triggers a replay attempt. Entries older than 24 hours are discarded — a stale 'I showed up' signal would be misinformation for a sponsor.

Rate limiting

Per-token limits: 30 polls/min, 10 presence signals/min, 5 check-ins/min, 20 messages/min. Four separate in-memory maps, pruned every 5 minutes alongside channel GC. A misbehaving client only affects its own channel.

— STORY 6 —

The Insights Engine

Three patterns, computed entirely on-device, gated by a confidence formula that is honest about its own limitations.

The three patterns

Phase mood correlation

For each circadian phase with at least 3 logs, compute the average mood score. If one phase consistently sits below 6 and the confidence score clears 72%, surface it. Example: 'Golden Hour tends to be harder. You've logged lower moods then 8 times.'

14-day mood trend

Compare average mood over the last 14 days against the 14 days prior. If the delta exceeds 1.2 points, note the direction. The 1.2 threshold prevents surfacing trivial fluctuations.

Check-in frequency

Which circadian phase does the user most commonly open the app in? If one phase accounts for more than 40% of all opens, note it. Habit pattern, not health observation.

The confidence formula

```
sizeScore = 1 - 1 / (1 + n/12)        // sigmoid — plateaus ~n=50
recency   = exp(-ageDays / 45)         // half-life 45 days
confidence = min(0.97, sizeScore × recency)

// Nothing surfaces below CONFIDENCE_THRESHOLD = 0.72
// Exported constant — independently testable
```

Honest empty states

hasEnoughData is false when fewer than 7 mood logs exist in the last 90 days — shows 'Log for a few days and patterns will appear here.' dataAgeDays > 30 shows 'Based on your logs from X days ago. Log today to refresh.' The engine is honest about its own data quality.

Zero network calls

Search useInsights.ts for fetch — zero results. The pattern detection reads from Sanctuary only. The data never moves.

— STORY 7 — NEW IN V2.3 —

End-to-End Encrypted Messages

This was planned for v3.0. It shipped in v2.3. The sponsor channel now supports text messages that the relay server cannot read.

The problem it solves

Presence signals (water, light) and check-ins are enough for some sponsor relationships. But a sponsor sometimes needs to send a message — 'I'm proud of you', 'call me when you're ready'. Without E2E encryption, those messages would travel through the relay server in plaintext. That would contradict the privacy architecture. So the messages are encrypted before they ever leave the device.

The cryptographic primitives — utils/crypto.ts

The library is tweetnacl — the JavaScript port of Daniel Bernstein's NaCl, the same cryptographic foundation used by Signal and WireGuard. It implements two operations:

Key generation — X25519 Diffie-Hellman

When a user or sponsor connects, generateKeyPair() calls nacl.box.keyPair(). This produces two 32-byte values: a public key (safe to share) and a secret key (never leaves the device). The math behind X25519 makes it computationally impossible to derive the secret key from the public key.

```
// utils/crypto.ts — generateKeyPair()
const kp = nacl.box.keyPair();
return {
  publicKey: encodeBase64(kp.publicKey),  // safe to share via relay
  secretKey: encodeBase64(kp.secretKey),  // stored in Fortress only
};
```

Message encryption — XSalsa20-Poly1305

nacl.box() combines X25519 key agreement with XSalsa20-Poly1305 authenticated encryption in a single operation. To encrypt a message for someone, you need: their public key and your secret key. The result is a ciphertext and a random nonce. Only someone with the corresponding secret key can decrypt it.

```
// utils/crypto.ts — encryptMessage()
const nonce = nacl.randomBytes(nacl.box.nonceLength); // random per message
const box = nacl.box(message, nonce, recipientPK, mySK);
return { ciphertext: encodeBase64(box), nonce: encodeBase64(nonce) };

// utils/crypto.ts — decryptMessage()
const decrypted = nacl.box.open(box, nonceBytes, senderPK, mySK);
if (!decrypted) return null; // tampered or wrong key — rejected cleanly
```

How keys get to each party — the key exchange

Each side generates a key pair when they connect. The public key is posted to the relay server via POST /api/sponsor/pubkey. The relay stores it in the channel (userPubKey or sponsorPubKey). The poll endpoint returns the peer's public key (peerPubKey) in the poll response. Each side stores the peer's public key in Fortress.

The relay server acts as a bulletin board for public keys — it can see them, but public keys are designed to be public. The private keys never leave the device. The server cannot compute the shared secret from two public keys alone.

```
// Key exchange flow
User connects  → generateKeyPair() → POST /pubkey (public key only)
Sponsor connects → generateKeyPair() → POST /pubkey (public key only)
Poll response  → peerPubKey delivered to each side
Both sides now → hasPeerKey = true → messaging enabled
```

What the server stores and what it never sees

Server stores	Server never sees
userPubKey (X25519 public key — 32 bytes)	userSecretKey — never leaves the device
sponsorPubKey (X25519 public key — 32 bytes)	sponsorSecretKey — never leaves the device
ciphertext + nonce (opaque bytes — cannot be decrypted without private key)	Plaintext message content — the server is a blind relay

Key lifecycle — what happens on disconnect

When disconnect() is called, Fortress.clear() deletes all 8 keys — the 5 channel credentials and the 3 crypto keys (CRYPTO_PK, CRYPTO_SK, PEER_PK) — in a single Promise.all over Object.values(FK). No private key survives a disconnect. The message history is cleared from local state. A new connection generates entirely new key pairs.

Q: "Can you read your users' messages?"

A: "No — and this is verifiable. The server stores only ciphertext and nonce. The private keys are generated on each device and stored in the OS Keychain — they never reach the server. To decrypt a message you need the recipient's private key. We don't have it. The server doesn't have it. It never leaves the device. The source is on GitHub — anyone can verify this."

— SOURCE CODE REFERENCE —

Key Code Excerpts

utils/crypto.ts — The complete E2E crypto layer (55 lines)

The entire cryptographic implementation. Three functions, no dependencies beyond tweetnacl. Each function is pure — no side effects, no state.

```
import nacl from 'tweetnacl';
import { encodeBase64, decodeBase64, encodeUTF8, decodeUTF8 } from
'tweetnacl-util';

generateKeyPair()   → nacl.box.keyPair() → { publicKey, secretKey } (base64)
encryptMessage()    → nacl.randomBytes(nonce) + nacl.box() → { ciphertext, nonce }
decryptMessage()    → nacl.box.open() → string | null (null = tampered or wrong
key)
```

utils/storage.ts — Fortress crypto keys (lines 39–47, 447–467)

```
// FK constants — all 8 Fortress keys
const FK = {
  CHANNEL_ID: 'kataleya.fortress.sponsor_channel_id',
  TOKEN:      'kataleya.fortress.sponsor_token',
  ROLE:       'kataleya.fortress.sponsor_role',
  LAST_POLL:  'kataleya.fortress.sponsor_last_poll',
  INVITE:     'kataleya.fortress.sponsor_invite',
  CRYPTO_SK:  'kataleya.fortress.crypto_sk',   // ← own private key
  CRYPTO_PK:  'kataleya.fortress.crypto_pk',   // ← own public key
  PEER_PK:    'kataleya.fortress.peer_pk',     // ← partner's public key
} as const;

// clear() — atomically deletes all 8 keys
async clear(): Promise<void> {
  await Promise.all(Object.values(FK).map(fortressDel));
}
// Object.values(FK) picks up any future keys automatically — no manual list
```

utils/hapticBloom.ts — Sponsor presence haptics

```
// Sponsor Water — 3 expanding ripple taps
sponsorWater(): Light(t=0) → Light(t=100ms) → Light(t=300ms)

// Sponsor Light — lub-dub heartbeat at 72 BPM
sponsorLight(): Medium(t=0) → Light(t=100) → Medium(t=380) → Light(t=480)
// Two complete cardiac cycles — perceptible through fabric, then silence

// Orchid bloom profiles
bloomSeed()      → silent (potential, waiting)
```

```
bloomBudding()   → single Light tap (Day 1)
bloomParting()   → two Light taps 180ms apart (Day 7)
bloomLabellum()  → Medium + Light echo 120ms (Day 30)
bloomFullBloom() → three Success pulses 130ms apart (Day 90)
```

— DUE DILIGENCE PREPARATION —

Questions You Will Be Asked

Question	Answer
How is this HIPAA compliant?	We are architecturally exempt. HIPAA applies to Covered Entities that handle Protected Health Information. Because health-sensitive data never leaves the user's device and never reaches our servers, we do not hold PHI. We cannot be subpoenaed for data we do not have.
Can you read your users' messages?	No. The server stores only ciphertext and nonce. Private keys are generated on each device, stored in the OS Keychain, and never reach the server. Without the recipient's private key, the ciphertext is unreadable. The source is on GitHub — anyone can verify this.
What happens if your server gets hacked?	The breach contains no health data and no message plaintext. The server holds channel routing state (check-in dates, orchid labels, milestone counts) persisted to a local JSON file, plus in-memory presence signals and encrypted message blobs. A breach would expose nothing clinically meaningful and no readable messages.
How were you built? Did AI write this?	I built this with AI assistance — primarily Gemini on Replit — and I understand every architectural decision in the codebase. The privacy architecture, vault separation, crypto choices, E2E messaging design, and burn ritual are all decisions I can explain and defend. AI wrote the code; I directed every decision.
Why not use Firebase or a cloud database?	Because our privacy promise would become a policy instead of a fact. Firebase stores data on Google's servers — a breach, court order, or API mistake could expose user data. Our architecture makes that impossible by design. There is no server that holds health data.
How does the backup work if you don't store the passphrase?	The passphrase derives a cryptographic key via PBKDF2 (100,000 iterations), then exits scope. The key encrypts the data. The key is never stored. Only the encrypted blob is saved. Without the passphrase, the blob is opaque to everyone — including us.
Can you add analytics later?	Not to health data without breaking the architecture. We could add product analytics (tab visits, session length) with no health data. But mood scores, journal content, and circadian history are structurally isolated in Sanctuary — adding a network call there requires deliberately breaking the vault separation, which would be immediately visible in code review.
What is your data retention policy?	For health data: we have no retention policy because we have no data. For server data: channel routing state is persisted to a local JSON file and expires automatically (48h unconnected, 90d connected). Message queues and presence signals are in-memory only — not persisted. Message queues are bounded at 200 entries per channel. No user database. Nothing to query.

— REFERENCE —

Glossary

Term	Plain-English definition
AES-256-CBC	The encryption algorithm used for backups. 256-bit key, CBC mode. The same standard used by banks and governments.
AsyncStorage	React Native's key-value store. Similar to localStorage in a browser. Encrypted at OS file level when device is locked.
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator. /dev/urandom on iOS, SecureRandom on Android. Used for encryption salts, IVs, and nonces.
Diffie-Hellman (X25519)	A key exchange algorithm. Two parties can compute a shared secret using only their own private key and the other's public key — without the secret ever being transmitted. Used for E2E message encryption.
Expo Router	The navigation system. File names in app/ become URL routes. app/(tabs)/sponsor.tsx becomes the /sponsor tab.
Fortress	The hardware-backed vault. iOS Keychain (Secure Enclave on modern devices) / Android Keystore. Holds 8 keys: 5 channel credentials and 3 E2E crypto keys.
HMAC-SHA256	Hash-based Message Authentication Code. Signs data so tampering is detectable. Used in backups to authenticate ciphertext before decryption.
JSI	JavaScript Interface. Lets JavaScript call directly into native code (C++/Swift/Kotlin) without a message bridge. Used by react-native-quick-crypto for fast native crypto.
NaCl / tweetnacl	NaCl (Networking and Cryptography Library) by Daniel Bernstein. The same cryptographic foundation used by Signal and WireGuard. tweetnacl is the JavaScript port. Used for E2E message encryption.
nonce	Number used once. A random value generated fresh for every encrypted message. Prevents replay attacks — the same plaintext encrypts to different ciphertext every time.
PBKDF2	Password-Based Key Derivation Function 2. Runs a passphrase through SHA-256 100,000 times to produce a cryptographic key. Makes brute-force attacks impractical.
RMS	Root Mean Square. Squares each value, averages the squares, takes the square root. Used for restlessness scoring because it weights sustained movement more than momentary jitter.
Sanctuary	The local-only SQLite vault. Holds all health data — mood logs, journal entries, circadian events. No network port. No user_id column.
Secure Enclave	A physically separate chip inside modern iPhones (A7 and later). Manages cryptographic keys independently of the main processor. Even Apple cannot extract keys stored there.

Surface	The AsyncStorage vault. Holds preferences and state — name, sobriety date, onboarding, notification prefs, tombstone, pending check-in, presence log, dark mode override.
timingSafeEqual	A constant-time comparison function. Always takes the same amount of time regardless of where values differ. Prevents timing side-channel attacks on MAC verification.
tombstone	A crash-recovery signal. Written to Surface before a destructive operation. If the process is killed mid-wipe, the next launch detects it and completes the wipe.
WAL mode	Write-Ahead Logging. SQLite setting that allows reads during writes without blocking. Used in Sanctuary for better performance.
XSalsa20-Poly1305	The authenticated encryption algorithm inside nacl.box. XSalsa20 is the cipher, Poly1305 is the authenticator. Encrypts and authenticates in one step. Each message uses a fresh random nonce.